Step 2 — Wrap It as a Single MCP Tool — Solution

> Back to the step · Hint 1 · Hint 2

Story: this exposes the whole Step 1 pipeline as one single `deep_research(topic) -> str` MCP tool. The tool's body is deliberately just "run the pipeline, return the summary" — the thinnest possible MCP layer over already-proven logic. **If not:** putting real logic in the tool body itself (timeouts, formatting) would mix the MCP layer's job with the pipeline's own job, exactly the mistake this project warns against.

`mcp_server.py` (new, at the repo root)

Story: this is an `MCPServer` with exactly one tool registered. The model and config are built once at module level, not inside the tool function, so each call just runs the pipeline against the already-built model and returns the final summary. **If not:** rebuilding the model client and config fresh on every single call would be slow and unnecessarily noisy.

```
# mcp_server.py  (Step 2 version — the thinnest wrapper)
# Exposes the whole Researcher/Fact-Checker pipeline as ONE MCP tool. The
# tool body just runs the pipeline and returns the summary. Logging goes to
# stderr (src/utils/logger.py); nothing prints to stdout -- stdout is the
# MCP protocol channel. Run it with:  mcp dev mcp_server.py
import sys

from mcp.server.mcpserver import MCPServer

from src.config import load_config
from src.models.llm import get_model
from src.pipeline import run_pipeline
from src.utils.logger import get_logger

CONFIG = load_config()
logger = get_logger(__name__)

mcp = MCPServer("deep-research-service")

@mcp.tool()
def deep_research(topic):
    """Research a topic through a fact-checked, multi-round pipeline and
    return a verified summary. One call runs a Researcher and a Fact-Checker
```

```
    until the summary is approved or a round cap is reached."""
    logger.info(f"deep_research: {topic}")
    result = run_pipeline(get_model(CONFIG), topic, CONFIG.max_revision_rounds)
    return result.final_summary

if __name__ == "__main__":
    print("deep-research service starting", file=sys.stderr)
    mcp.run(transport="stdio")
```

Trying it

With the MCP Inspector:

```
mcp dev mcp_server.py
```

Then call `deep_research` with `topic = "Retrieval-Augmented Generation (RAG)"`. You get back the plain final summary — the fact-checked one, after the loop ran — with **no trace yet**. A caller sees a single clean answer and has no idea a two-agent revision loop produced it. That's the whole "agent-as-a-service" idea working.

What's deliberately *not* here yet

- **No trace.** The caller waits on a multi-round call with zero feedback, then gets only the final string. That's a bad experience for a slow tool — Step 3 fixes it by returning a revision trace alongside the summary.
- **No timeout.** A slow or broken run hangs the caller indefinitely — Step 4 adds a timeout and a partial fallback.

Resist adding either now. Keeping Step 2 a two-line tool body proves the wrapping itself is trivial; all the interesting *design* is deciding what a slow, multi-step process should look like from the outside, which is Steps 3 and 4.

Tests for this step: the integration test (Step 4's file) confirms a real `deep_research` call returns a non-empty summary — proof the pipeline runs correctly through the MCP layer.